The Driver's Assistant: Utilising Synthetic Data Generation and Deep Learning for Traffic Sign Classification

Author:
Alexandros G. Stergiou

Project Supervisor:
Dr. Adrian Clark

Submitted to University of Essex Department of Computer Science and Electronic Engineering as part of the requirements of the MSc degree in Advance Computer Science.

August 25, 2017

Abstract

A main concern of road sign recognition systems has been the use of a large dataset obtained through research and manually finding sufficient data in order to conduct the necessary experiments and optimally, create a model with high accuracy rates. The purpose of this dissertation is the introduction of a traffic sign recognition system , which is based on synthetic data generation methods (SGTD), that is not bound to a specific set of road signs as occurring in a country or region, but can generate training examples given a defined set of templates. This work can significantly decrease the time spent by researches or system developers in order to acquire data and allows the reallocation of the main project focus on the construction of the predictive model. Although, the training dataset is synthetically created, the variations in visual appearance between examples, such as differences in illumination and rotation, are reflected on each image to best represent road signs in real world settings and have been implemented using Computer Vision methods. The main classifier that the system is based on is a Convolutional Neural Network (CNN) that can achieve close to state-of-the art accuracy results when trained on actual data.

Contents

1 Introduction **3**

2 Related Work **5**
2.1 Previous systems and varieties of methods used 5
2.2 Current efforts and state of the art 6

3 Dataset **7**
3.1 Synthetic data resources 7
3.2 Data generation based on templates and backgrounds 8
3.3 Dataset sampling, validation set collection and format 12

4 Classification Methodology **15**
4.1 Convolutional neural network and using three dimensional space 15
4.2 Utilising exponential linear units 17
4.3 Performing normalisation at each training batch 18
4.4 Sample-based discretisation process 20
4.5 Regularisation of each layer 21

5 Experiments and Results **23**
5.1 Background of experiments 23
5.2 Classification results 25

6 Summary, Conclusions and Future work **28**
6.1 Summary and Conclusions about the project 28
6.2 Future work and improvements 30

References **33**

1 Introduction

The classification of road signs has been a challenging field in both the academic world as well as in the industry, with the latter receiving high interest. The direct applications of such systems can range from urban environment understanding to also being an integral part of Advance Driver Assistance Systems (ADAS) [1][2]. The systems are able to warn or guide the driver by using the extracted information, provided by the road signs (as well as in more advance systems that also include the understanding of additional data from the road, the car distances, etc.). The essential problem of this task focuses on the fact that each sign has a unique combination of shape, colour and graphics that act as a minimalistic representation of the environment and road conditions that enables the driver to understand and possibly prepare (if necessary) for what he may face. Signs are expected to be highly visible to drivers and signs that represent similar warning/guidance only display a minimum variance between one another. However because of their exposure to different conditions, instances occur where the signs are completely distorted and the process of understanding what is depicted becomes challenging for both human and machines. Most of the classification shortfalls are because of differences in the viewpoints of the observer, the lighting of the object, obstacles that do not allow the viewing of the entire sign and the environment in total (for example the contrast between dusk and dawn), colour fading or even motion blur [3].

Figure 1: Examples of traffic sign distortions from the validation set created. Form left to right: example of obstacle in the image, motion blur, colour fading, lighting variations and shade from leaves and environment conditions in combination with low lighting levels.

A variety of approaches and methodologies have been used for the first named challenge in the field. Most common approaches in the classification problem include Support Vector Machines (SVMs) [4], Convolutional Neural Networks [5][6] and Random Forests [7] among others. The features that were the base for many of these methods were the shape and colour of the sign class, as simple thresholding techniques or background removal approaches, can be a valuable initial indicator that can be utilised by for the categorisation procedure. Therefore, the level of precision of the classifier is strongly associated with the features that exist, with minimal variations across the same class of signs, obtained in different instances.
The second challenge is the time and effort necessary in order to obtain a large dataset that can be used to train the necessary classification model. An example of the time con-

sumption required could be seen in the German Traffic Sign Recognition Benchmark [8], which is composed of 43 distinct classes and approximately 50000 images, that were extracted from 10 hours video, with a frame rate of 25fps. Another instance, where manual work was required to construct the dataset used, was an image segmentation and road sign detection and classification system using SVMs for Spanish signs, by S. Maldonado Bascón [9]. The total number of examples that were included in the final work reached 35714, with 153 distinct classes present in the dataset. Other datasets used in recognition or detection project include the Swedish roads dataset obtained from a 20000-frame video [10], as well as the Belgian traffic sign dataset from KU Leuven [11]. Because of the different subset of traffic signs that are used by each dataset, it has been difficult to benchmark the classification stage of each of the methodologies used. Many systems may focus on specific categories such as speed limits, which are mostly comprised of circular signs with red lining or warning signs with triangular shape.
One of the main purposes of this work is to provide an effective evaluation method for each algorithm by comprising a large number of classes that can be user defined and unbounded to specific road sign database. Apart form using a generator to create the synthetic data, the work also includes a Deep ConvNet that is created with using advance activation functions and also describes how three different activation functions behaved during tests. The activations that were tested on the network included LeakyReLu [12], Parametric Rectified Linear Unit (PReLU) [13] and Exponensial Linear Units (ELUs) as described by Djork-Arné Clevert [14]. Moreover, some early testing was done the with the recent scaled exponential linear units (SELU) [15], however the results were not completed and therefore, not presented in this thesis. Also different network architectures were tested and the one described in this paper yields the best results.
The document is organised as follows: Section 2 displays an overview of the relative work produce in the field of traffic sign classification and synthetic data generation. Section 3 describes the methodology used to create the complete dataset of synthetic examples from which the training data was sampled from. Section 4 is an in-depth analysis of the CNN that was used and the structure that was followed for creating the network. The variety of experiments that were conducted as well as the results are located in Section 5. Last but not least, Section 6 serves as the summary of the work produced during this project and the future work required for improving the system.

2 Related Work

2.1 Previous systems and varieties of methods used

There are several distinct approaches and publications for traffic sign recognition as it has been an active research field in Computer Vision and Machine Learning. The standardised approach would include using Computer Vision techniques for pre-processing the images in such a way that the similarity between the classes is minimised drastically, while the similarity within a class should be maximised. One of the earliest works in the field was the work of Akatsuka with his paper that was centered towards using a colour processing system to initially locate the road sign in the image and later a template matching method to allow the real-time extraction of the road sign [16]. Another example of the first works and the limitations of those times, was another attempt [17] that utilised an optical filter as the systems capacity was highly restricted and therefore, high amounts of data were unusable. Because of the increase in computational power and the advances in both fields of Machine Learning and Computer Vision, most of the projects used for the recognition and classification of traffic signs are implemented based on two major categories.
The first would include using Support Vector Machines. This combination is primarily partnered with the use of Histogram of Oriental Gradients (HOG) features [18][19] or with the use of a Gaussian kernel. However all these approaches include common image analysis in the pre-processing stage, as the main steps that include colour thresholding and shape detection, vary little between the methods. The notable advantage of using SVMs, than other classification methods, is the fact that they are easier to train and they are additionally less prone to over-fitting as regularisation in used. Thus, even in occasions in which the number of attributes is larger than the number of observations, the kernel parameters (if tuned properly) do not allow over-fitting to occur. By creating an SVM that is independent of the dimensionality of the feature space, while being dependent to the margin, they become close related to the regularisation parameters used.
The other approach would be the use of Convolutional Neural Networks, which are based on the human perception of objects and they include three main principles in their structure: sparse interactions, parameter sharing and equivalent representations [20]. Although traditional artificial networks are composed of layers and use matrix multiplication (as there are several parameters that each describe the interaction between each input and output), CNNs have *sparse weighs* by making each kernel smaller than its input. This yields and output that consequently requires less operations. Most CNNs also include a pooling layers throughout the network to lower the image dimensionality [3], which allows for a robust representation of the image without the inclusions of distortion shifts. The variations in size and the number of neurons per layer of even the construction of a multi-network system [22], is based on the difficulty of making a prediction for the task and also the training sample available.

2.2 Current efforts and state of the art

The area of traffic sign recognition has been matured with the increase demand of stable and better-than-human performance systems. One of these first systems was introduced in the 2012 paper by Dan Ciresan that describe the use of a method that combined the outputs of multiple Deep Neural Networks called Multi-Column DNN (MCDNN) [22]. Although kernel filters are used in the same was as seen in CNNs [23], the essential difference is the use of MaxPooling layers [24] instead of sub-sampling. The structure chosen is based on convolution layers in combination with the MaxPooling layers in which examples are only down-sampled by one pixel. The final results are calculated based on the average, produced by every neural network incorporated in the systems structure. The averaging of single network outputs for this task followed the same principle as they were described by Ueda and Meier [25][26]. Through experiments made on digit recognition, by averaging the outputs of multiple DNN, it allowed the method to better generalise on the test set. The accuracy of this approach achieved recognition rates that were ranked better than the average human with the MSDNN performing at 0.9946 accuracy, while a single DNN attained 0.9852 with a margin of $\pm$ 0.15%.
Although the MSDNN has shown improvement from other systems, its size and needs (as it was trained with the use of four graphics cards), constitute it as a voluminous implementation for a task that has only a certain degree of complexity. An alternative technique as suggested by Junqi Jin [6] was the utilisation of a hinge loss cost function similar to the one that it is used to train margin classifiers (SVMs). In such systems, the main strategy that is followed, is to maximise the margin while minimizing the average loss. Therefore, the basic element would be the introduction of a loss function where the penalty is proportional to the predicted labels ”distance” from the margin boundary between the classes. In this work, the learning stops when the margin is above zero and training example has been rightfully classified. On the other hand, if the margin is not big enough or the training example has been incorrectly classified, the learning procedure continues.

$$cost = I(y > 0) \cdot max(0, 1 - \hat{y}) + I(y < 0) \cdot max(0, 1 + \hat{y}), \tag{1}$$

Where y represent the actual label of the example and $\hat{y}$ represents the predicted label.

Albeit the accuracy improvements that both of the previous solutions, to the classification problem, provide, the system that achieves the best solutions currently was presented in the 2016 paper by Haloi [27]. The novelty of this work is that the network includes spatial transformer layers and an inception module that is used to extract local and global features in the images. The results of this method were eveluated on the GTSRB dataset and achieved a state-of-the-art accuracy rate of 99.98%. A modified version of GoogLeNet [28] was used in this task in combination with spatial transformer layers that also used backpropagation for the training. This layer eliminated the need for data augmentation in the CNN while allowed it to learn the active transformation of features map.

3 Dataset

3.1 Synthetic data resources

As it has been mentioned, the dataset used for training the system has been created programmatically in order to account for the large number and variety of traffic sign sets that can be used for the recognition system. For the purposes of this work, the basic templates that were used were acquired from the YouGov [29] website. From these, because of the large times required the final set was downscaled to the fifty most significant and common classes as seen in Fig.2. The original set included more than 600 road signs. The grouping of the signs is presumed to be the same as the one found in the government publication [30]. The three main categories from which training set data was sampled from are:

- Warning signs: These signs are triangular shape with red borders. The rotation of the triangle can vary in one instance in which it is used to show the priority in an intersection. In rural roads road signs also show the type of wildlife present as well as the danger of animals crossing the road unexpectedly.
- Regulatory signs: They are usually found to have a circular shape, however their colours may vary (direction indicators or vehicles restrictions). Their main role is to denote the rules that apply on the road. In contrast with other parts of the world, most of the European countries distinguish regulatory signs from speed limit signs as two distinct categories, with the former limited to specifications for the direction to be followed and the type of vehicle that can access the road, while the latter includes the maximum and minimum acceptable speeds.
- Speed limits: They are circular with a red lining (for maximum speed), or blue (minimum). The purpose of this category as the name suggest is to establish the speed range that a car can be moving in.

Figure 2: List of all the traffic sign classes that were used. This data can be changed, however for the purposes of the project the possible classes will the 50 displayed

The second set of resources that were used and were essential for synthesising the training set, was composed of 1000 images of British roads (and a small portion of general examples from other countries). These images were used to represent the background to be utilised once the image generation process commences. There were 500 examples of urban areas and roads, while the other half of the backgrounds were depicting rural

environments. This separation was done in order to account the different variations of environments that a sign can be seen. Additionally, some sign classes are more probable to be located in rural areas, such as wildlife warning signs. On the contrary, other signs, for example intersection warnings, may be more appropriate for urban environments. In this implementation of the generator procedure, signs were generated for all the 1000 background images to thus increase the size of the dataset without considering the sign class and the probability that, that class may be more suited for a specific scenario (urban,rural,highway). It is possible to do this however, with the aid of a human expert and the correct labeling of each image with the cost of generating less examples than the current procedure.
The backgrounds were found from Google Search and capture different scenarios during both daytime and dusk. The weather conditions also differ between examples, as there are many in which weather can be; cloudy (heavily clouded and thus more grey or slightly clouded with some openings) or sunny. There are instances in which the sky is not visible and only walls or buildings are present, while others depict the sky with almost half of the image dedicated to showing the weather conditions. Another important aspect of the background is the clarity of the environment, for example a highly humid domain will show a greater amount of distortion in object than a dry atmosphere. All these parameters were essential in order to create a significantly large dataset with which the method is trained.The sizes and file formats of the pictures were not considered and were different for each image, as they have been directly downloaded from the Internet without any preprocessing. This achieves an additional level of distortion as the aspect ratio and size of the images will change later, once the main example generator processes the files.

3.2 Data generation based on templates and backgrounds

The initial step in the main program is the segmentation of the traffic sign from the background. The background of the templates is usually a variation tone of white or black therefore, the process that was proposed was to find the borders of the traffic sign in the image and segment it by storing the valuable part of the picture into a new directory, in a PNG format to account for RGBA. It was chosen to use RGBA as it also includes an alpha channel that shows the opacity of a pixel and therefore can be manipulated in order to effectively blend/fade in the final step of the process, in which the smooth marriage between the foreground and background is achieved. The simple equation that was used to extract the sign can be found in (2) for the left border and (3) for the right, for each row of pixels (y_i) in the image.The threshold values that were chosen were found from continuous test and experiments that were made in order to minimise the number of non-foreground pixels that were obtained during the segmentation of the sign. Programmatically, pixels are defined as one dimensional (grey), if they show a difference of

lower than 15 between each possible product from the subtraction of two channels. Under other circumstances this difference would be too great for a variety of colours, however because signs are assigned with bright colours, in order for them to be distinguishable in a plethora of environments, the assumption of large channel difference yields satisfying results. This mindset is also applied to the second part of the process in which each pixel is compared with its neighbour to find where a large variation exists. A good indication for a pixel being a border pixel of the sign, would be the difference in one (or more than one) of its channels compared to the pixel that is next to.

$$leftborder = (x_j, y_i) : \alpha_{(x_j,y_i)} = \begin{cases} |r_{x_j,y_i} - g_{x_j,y_i}| \geq 15 \\ |r_{x_j,y_i} - b_{x_j,y_i}| \geq 15 \\ |b_{x_j,y_i} - g_{x_j,y_i}| \geq 15 \end{cases} \text{,where} \begin{matrix} |r_{x_j,y_i} - r_{x_{j-1},y_i}| \geq 20, or \\ |b_{x_j,y_i} - b_{x_{j-1},y_i}| \geq 20, or \\ |g_{x_j,y_i} - g_{x_{j-1},y_i}| \geq 20 \end{matrix} \tag{2}$$
$$\text{for every } 0 \leq y_i \leq img_{height}, \text{ and } 1 \leq x \leq img_{width}$$

$$rightborder = (x_k, y_i) : \beta_{(x_k,y_i)} = \begin{cases} |r_{x_k,y_i} - g_{x_k,y_i}| \geq 15 \\ |r_{x_k,y_i} - b_{x_k,y_i}| \geq 15 \\ |b_{x_k,y_i} - g_{x_k,y_i}| \geq 15 \end{cases} \text{,where} \begin{matrix} |r_{x_k,y_i} - r_{x_{k-1},y_i}| \geq 20, or \\ |b_{x_k,y_i} - b_{x_{k-1},y_i}| \geq 20, or \\ |g_{x_k,y_i} - g_{x_{k-1},y_i}| \geq 20 \end{matrix} \tag{3}$$
$$\text{for every } 0 \leq y_i \leq img_{height}, \text{ and } img_{width} \geq x \geq 1$$

Once the template is stored in an RGBA format the *Affine Transformation* can be computed [31]. This method is used to represent both changes in the rotation and different scales of the sign. This has been a standardised technique for producing the illusion created by non-ideal camera angles that are focused on filming specific objects [32]. Geometric distortions are an essential aspect of real examples, as the position of the camera is very likely to produce perspective irregularities while altering the entire scene. The three main uses in this system is the scaling, translation and rotation functions that it provides and their equation can be found in (4). Affine transform falls into the class of linear 2D geometric transformations in which variables in the original image (such as the colour space values of a pixel P(x,y)) are linked to new variables (for pixel P'(x',y')). Therefore, the pixel grid of the image is deformed and mapped to a destination image. For this work, twenty different transformations were used, all based on three significant points in the image. The first point is located close to the start of the axis at the one tenth of both the height and the width, or alternatively in some instances, it is at the right side of the image. This is the point that determines the shearing of the final image, based on the point that it is assigned after the transform is performed. Secondly, the other two points are primarily used for both the rotation and scaling of the image. One is placed at the top in the middle of the image. The second of the two is defined as in the middle of the left side, or the equivalent in some cases, at the right side. All the twenty distinct affine transforms are applied to each image class and are stored in files on a directory that is based on the sign that the transformation was applied to.

$$\begin{bmatrix} x_2 \\ y_2 \end{bmatrix} = A \times \begin{bmatrix} x_1 \\ y_1 \end{bmatrix} + B \tag{4}$$

Following the creation of the different transformations, the system will then compute the average and the root mean square (RMS) brightness of the pixels in the image. The reason for choosing to consider both a measure of variability and the measure of the magnitude of a set of numbers, is because the RMS could only be bigger or equal than the average, which will relate to a higher number assigned to the brightness of the image. However, considering the average pixel values, the image may be assigned a brightness value that is not entirely representative. This might be due to the fact that a minority of pixels include values that are in the borders of the distribution [33] and thus tempering with average value. A possible method to overcome the problem is with the use of the median value. Nonetheless, it has been observed that this difference was beneficial for the method as it was effectively representing variations that are shown in natural settings and the discrete material that the sign is made with. Therefore, in order for preserving a portion of the image noise that is seen in real photos, the average and the root mean square values are more beneficial for the creation of the dataset.
In addition to the average and RMS values, two different practices are used for understanding the brightness. The first uses a greyscaled version of the image and computes the average and RMS pixel values. This is considered a standardised method, as the luminance in this instance is defined as the single dimensional image, without the use of the information from any of the other three image channels. An alternative method is the utilisation of the HSP colour model [34]. The major difference from other colour models such as HSV (which uses the V component to demonstrate the brightness as the maximum value from the RGB channels), or HSL (with L being the average of the maximum and minimum value from the RGB channels), is the fact that it includes the degree of influence of each RGB channel. The equation used for the conversion can be seen in (5), in which the constants represent the extend to which each colour affects the pixel's luminance. This model was partially based on the function that Adobe Photoshop uses to translate the brightness when converting a coloured image to a greyscaled.

$$brightness = \sqrt{0.299 \times R^2 + 0.587 \times G^2 + 0.114 \times B^2}, \tag{5}$$

The same process is also applied to the image with the RMS and average values computed for the 1D image and the perceived brightness. At the stage in which all the necessary values have been found for both the background and foreground, the margin of the two is calculated as the absolute difference of their respective values. Then in order to find the brightness that the sign needs to be adapted to, the margin is divided by a ratio that was based on maximising the similarity of all the image features while adding colour distortion to represent background conditions. For similarity measure and to test the thresholds, the images were segmented in ten distinct regions. For each of them a histogram was plotted. The histograms were later compared with their equivalent regions at the sign and at the background, as it was found that the best results (histograms that maximised the similarity with the foreground and background images) came with the use of the thresholds selected

for each category. The outcome of each approach can be seen in Fig. 3.

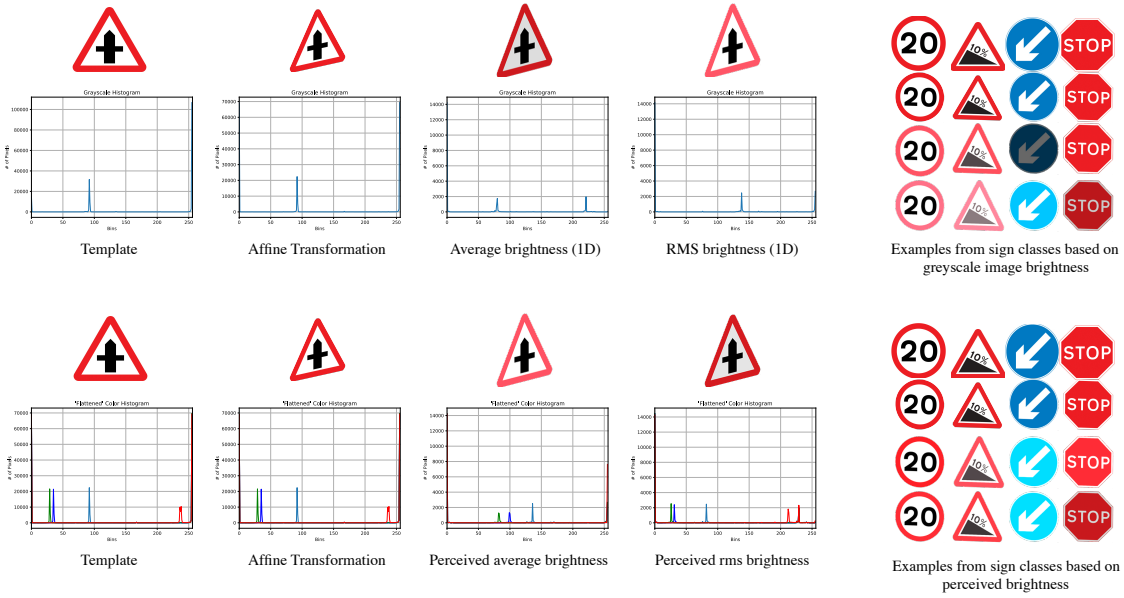

Figure 3: Difference in brightness by using dissimilar values and methods to compare the foreground and background variations. The top row consists the template image, one of the twenty affine transforms (number 15) and the two new images produced by comparing the average and root mean square values of the background and the sign, utilising the 1D approach. The bottom row includes the two examples based on the perceived brightness of the foreground-background. There is also another range of examples from the methods.

As a final step, the system will store the image by applying the new illumination that was found through using an enhancer function. There are examples in which this automated procedure was deemed insufficient and the distortions overshadowed the image features, as seen in Fig. 4. For that reason, the generated signs are revised with a set of threshold values to segment valuable examples from others that cannot be used. The data separation function considers both case in which the traffic sign may be black and white (such as class number 49) or coloured signs. In the instance of a grey sign, an additional function is used to store the file path of every image that is grey. This is achieved by calculating the difference between all the individual channels on the average pixel value of the image and reassuring that the margin is lower than one. Consequently, in the case that the image is indeed composed of only grey pixels it will be processed separately. A useful grey image that is not overly bright nor dark, which would have shown disfigured features, can be perceived within the values of 70-155. These limits are not fixed as they can be changed if required and they represent the general regions of the limits that can be used for the solution. On the other occasion, coloured sings are judged based on the difference be-

tween the RGB values. However, conditional independence is assumed between the three relationships, as for example a bight red image will have a great margin when comparing the Red and Blue channel but the difference may be less than one in the case of Blue and Green channels.
The total number of signs that is currently generated and can be used by the classifier for training are 2,388,397. At the final stage of the generator a text file is also created that holds information in regards to the examples per class as well as the total number of data created.

Figure 4: Examples of images that were generate based on the brightness of the background and foreground and are deemed unsuitable and less informative for training.

The last step of the synthetic example generator is the blending of the background and foreground. This action is performed by pasting the sign at the background, while also creating a blending effect at the border of the sign in order to create better representations. The bordering one pixel is manipulated in a way in which the opacity is decreased and therefore, the final RGB values are change to an intermediate result between the actual sign and the colours of the background. Initially the procedure is performed in 150x150 images as the blending can be done more effectively and with greater detail. Afterwards, the image is resized to a 48x48 size to minimise the divergence from actual images.

3.3 Dataset sampling, validation set collection and format

Because of the large number of images generated, the results and rates obtained were based on samples from the entire dataset. In order to create the final training set, stratified sampling was used on the created images. This technique allows the reduction of samples needed, without compromising the best precision that can be achieved by the classifier. It includes sampling each population of classes independently and therefore the sampling error is reduced. Additionally, sampling has been an effective way of estimating the overall accuracy of the method within less time as the increase of data also implies the greater requirements for computational power and running time. Furthermore, by randomly selecting data from each strata (class that has a certain number of generated examples), the distribution of examples and their differences between one another will be preserved in the new set, as stratified random sampling ensures the presences of important subgroups within the selected data. Moreover, relationships between the subgroups are maintained allowing also the sampling of extreme examples and thus creating a representative subset.

Lastly, the high statistical precision permits the decrease the size of the set and the hardware requirements for the system to store and run.
The validation set that was used to test the accuracy on actual images of the system, trained on synthetic data, were obtained through three different resources:

- Image files from Google. These examples were easy to acquire, however a limitation to this resource was found for classes that were not usual for many roads (such as the soft verges sign or many of the wildlife signs). In addition, images from Google did not include distortions such as motion blur that can be seen from video images or resolution inconsistencies, as most of the photographs were taken from professionals, with the necessary equipment. Therefore, some examples do not represent fully the conditions of the images and can be proven misleading when evaluating the classifier.
- Segmented images from video resources. An alternative to still images is the extraction of signs from videos on YouTube and other video hosting sites. This approach was resourceful, as the representation of a road sign in video is done in the span of several frames and thus can produce a substantial amount of examples. A constraint however, is the fact that in order for usable examples to be found, videos need to be of good quality and high definition in order to extract image samples from them. This was not proven to be a significant complication in this case however, because considering of the affordability of high resolution cameras, most video resources were either HD ready or full HD resolution.
- Google street view. The last and more important resource that was utilised was the 360 degree feature from Google. By virtually going through specific areas with distinct environments (the mountains of Wales while in search for falling debris signs or London and Birmingham for end of lorry ban zone signs), the time required for the acquisition of real examples was significantly decreased. Moreover, because of the multi-angle feature of the Google car, that captured these images, each sign is taken from several different angles providing (as with the video resources) many useful data. Also the motion blur exists in the examples, as the photos were taken from the camera mounted on the car.

The environment and illumination states of all these signs vary significantly as not only they are from different parts of the United Kingdom, but during different times of the day and with distinct weather conditions. It is also worth mentioning that instances exist in which the signs may be disfigured because of hardships of the climate or their placement on the road. Also, pictures exist that the signs may be partially depicted, that might be because of the surrounding vegetation or objects in from of them (such as other signs). All these occasions are good examples as a high performing system will show an understanding of signs in depth in order to be able to correctly predict the class and not include

noise as part of the class features (such as minor background patterns that may be found because of the environment of the majority of the category's sign).
For convenience and based on previous work for datasets with the German Traffic Sin Benchmark and the Belgian Traffic sign benchmark, both the validation set images as well as the synthetic generated training sample, are resized to 48x48 images and saved as JPG file format. The file format was changed form the original PNG that has greater analysis but suffers from the demand for larger storing space available. The difference can be seen in the fully synthetic dataset which has a size of approximately ten gigabytes when the JPG format is used while the size increases dramatically (approximately 11.2 times) when PNG is used. Although JPG is considered a lossy format, it is a necessary compromisation that is required to be made since the storing requirements of PNG images are much larger than the ones available. The last part of the normaliser script is responsible for applying a Non-Local Means Denoising that produces a blurred version of the image, by replacing the colour of each pixel with an average of similar coloured pixels. This filter has been proven to provide better results than the standard Gaussian filter, as allows both the effective denoising of the actual data in the validation set, as well tempering with the appearance of the synthetic data in order to minimise the features that do not exist in real examples. Also, by considering all the similar pixels in a region and assigning the average value, the colour similarity between the training and validation set increases significantly. Although the classifier is only concerned with a single dimension, the variations done in all the three channels of the images will also be represented in the grey-scaled version of the images as well. The kernel used to apply the filter is of size 5x5 in order to accomplish a box blur.

Figure 5: Examples of synthetically created images from the training dataset-SGTD (top row) and actual photos real life traffic signs from the validation set(bottom row).

4 Classification Methodology

4.1 Convolutional neural network and using three dimensional space

The main advantage of using Deep Learning and in particular CNNs, for image recognition than fully conected artificial neural networks, is the fact that they include 3D volumes in their neurons. This means that layers in ConvNets have neurons that include all three dimensions of the image (width, height and depth/colour) [3] . Each neuron in succeeded layers is only connected to a small region of the previous layer, instead of all the neurons connected with each other. CNNs are feed forward models, meaning that information only moves in one direction, similarly as it is done in the visual cortex. In the cortex, cortical neurons (regions of the mammalian brain) are responsible for a specific region, for what is perceived in the field of vision (receptive field) [37]. The overlapping of these fields is used to comprehend and perceive the entire scenario/environment. Translating this to a machine format, the network will start to learn each filter for each neuron based on the training examples that were provided, in contrast to other methodologies in which these filters were handcrafted. Therefore, neurons cannot form circles, which is the essential difference from structures such as Reccurent Neural Networks (RNNs).

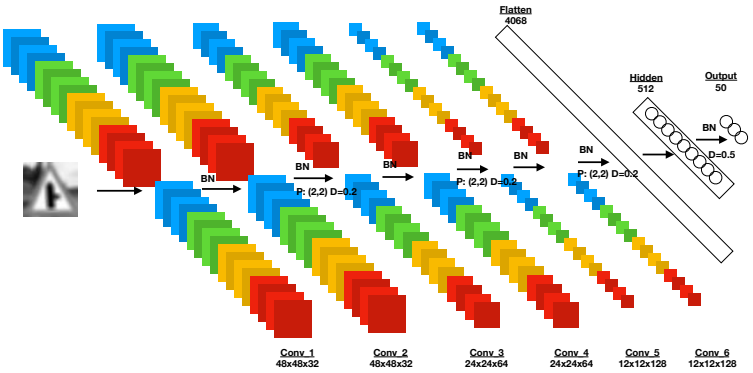

Figure 6: Examples of synthetically created images from the training dataset-SGTD (top row) and actual photos real life traffic signs from the validation set(bottom row).

The full architecture of the network can be visualised in Fig.6 and in Table I, which displays an analysis of the methods and parameters used for the construction of the system. The network that is introduced in this work consists of six convolution layers, with max pooling and dropout being used every two convolution layer, and a hidden layer (loosly based on similar projects from online resources[1]), that maps the outputs of the flatten

[1]The two repositories that were used as inspiration in the early implementation stages were the works of Sasank Chilamkurthy: https://github.com/chsasank/Traffic-Sign-Classification.keras and Tian Liang Yuan: https://github.com/hello2all/GTSRB_Keras_STN

layer, that consists the one-dimensional matrix of the images, to their respected classes in the output layer. The network maximises the multidimensionality aspect of the overall mindset of CNNs, which is equivalent to maximising the problem space that allows extracting the useful features easier. The Exponential Linear Units (ELU) [14] activation function is applied to the output of every convolutional layer along side with a batch normalisation. ELU is also used as an activation function at the fully connected layer in order to correctly assign each example to its class. The kernel sizes used in all convolution layers are of 3x3, as the size has been standardised and used in multiple projects [38]. It is worth to note that because the main purpose of this work is to create a fully flexible system, the number of output layers neurons can be changed in the instance that a different size of traffic signs templates are used during the image generation process. In this way the network can also run with customisable class sizes that can be defined by the user and will be based on the road sign categories that will be used. Also, the architecture of the network and the methodology that was followed was based on the fifty classes that were described in Section 3. Therefore, if a different set is to be used, with respect to the inherited difficulty of the classification of each class, the network may require some adaptations (for example the number of neurons in the hidden layer if the images are not of size 48x48, or the depth of the network if the number of classes is small and the variations in features between classes is great).

Table 1: A complete analysis of the CNN structure and the number of parameters of each component. The final line in the table is used to represent the total number of trainable and non-trainable (produced by batch normalisation) parameters of the network.

Layer (type)	Output Shape	Param #
input_1 (InputLayer)	(None, 48, 48, 3)	0
conv2d_1 (Conv2D)	(None, 48, 48, 32)	896
batch_normalization_1 (Batch	(None, 48, 48, 32)	128
elu_1 (ELU)	(None, 48, 48, 32)	0
conv2d_2 (Conv2D)	(None, 48, 48, 32)	9248
batch_normalization_2 (Batch	(None, 48, 48, 32)	128
elu_2 (ELU)	(None, 48, 48, 32)	0
max_pooling2d_1 (MaxPooling2	(None, 24, 24, 32)	0
dropout_1 (Dropout)	(None, 24, 24, 32)	0
conv2d_3 (Conv2D)	(None, 24, 24, 64)	18496
batch_normalization_3 (Batch	(None, 24, 24, 64)	256
elu_3 (ELU)	(None, 24, 24, 64)	0
conv2d_4 (Conv2D)	(None, 24, 24, 64)	36928
batch_normalization_4 (Batch	(None, 24, 24, 64)	256
elu_4 (ELU)	(None, 24, 24, 64)	0
max_pooling2d_2 (MaxPooling2	(None, 12, 12, 64)	0
dropout_2 (Dropout)	(None, 12, 12, 64)	0
conv2d_5 (Conv2D)	(None, 12, 12, 128)	73856
batch_normalization_5 (Batch	(None, 12, 12, 128)	512
elu_5 (ELU)	(None, 12, 12, 128)	0
conv2d_6 (Conv2D)	(None, 12, 12, 128)	147584
batch_normalization_6 (Batch	(None, 12, 12, 128)	512
elu_6 (ELU)	(None, 12, 12, 128)	0
max_pooling2d_3 (MaxPooling2	(None, 6, 6, 128)	0
dropout_3 (Dropout)	(None, 6, 6, 128)	0
flatten_1 (Flatten)	(None, 4608)	0
dense_1 (Dense)	(None, 512)	2359808
batch_normalization_7 (Batch	(None, 512)	2048
elu_7 (ELU)	(None, 512)	0
dropout_4 (Dropout)	(None, 512)	0
dense_2 (Dense)	(None, 50)	25650

Total params: 2,676,306
Trainable params: 2,674,386
Non-trainable params: 1,920

4.2 Utilising exponential linear units

A variety of activation methods were tested on the classifier, as it will be seen in the later chapter. The essentiality of an advance activation layer for the system comes based on the problem of using the standard Rectified Linear Units (ReLu) as activation function. These units are shown to be very sensitive during training. Especially in the case in which a large gradient changes the weights and causes the neuron to behave in such a way that it does not activate once new data is fed to the network. This is often called the "dying ReLu" problem and it has been an inspiration for the introduction of different activation functions over the years. When this problem occurs, the gradient of the neuron will "freeze" to a zero value from that point forward during the classifier's training. Although the ReLu rectifier is very fast to compute, since the function simply outputs zero when input is smaller than zero and essentially mimics the identity function. Since in the case of a less than zero input, the output is zero, it can be easy calculated that the rate of change will also be zero. In any other case the derivative will be equal to one as seen in (6).

$$\frac{d}{d_x} f_{(x)} = \begin{cases} 0 & if \quad x < 0 \\ 1 & if \quad x > 0 \end{cases} ,Not \quad differentiable \quad at \quad x = 0 \tag{6}$$

Therefore, relu cannot recuperate from this issue as not only the output will be zero as it was discussed previously, but also the derivative will have a zero value. Although this pitfall of the relu rectifiers has been the cause for researching alternative methods, an advantage that it provides is the sparsity of the representations that is fed to it during training. An example of this can be perceived through information disentangling, as in a highly dense representation, major changes in the input would resolve to large modifications in the representation vector appearance. If what is used alternatively include small, robust and sparse delineations, there is a conservation of the features, throughout iterations [39]. Additionally, sparse representations are more probable to be linearly separable since the multi-dimensional space of the examples can also effectively reflect the original data. Furthermore, considering the fact that different inputs may hold a different information size, than previous ones, using a shifting-size structure to obtain information may yield better results. By changing the number of active neurons at each step, the dimensionality of the model can be controlled and tailored to the needs of the data. However, nevertheless of these different factors, densely representations are shown to produce a greater amount of information, as they are exponentially richer than the selective sparse counterpart [39]. Both of these representations can be perceived as a trade-off for a system similarly to the precision and recall that can be found when computing the truth table of a system. For the needs of this model, rectified linear units were not used as the difference between the training and validation samples were great enough to an extend in which, a sparse representation would not include examples with features that are better distinguishable than what would have been seen in a densely structure. This is due to the synthetic nature of the training data and thus, the need for a higher complexity system with a greater number of

parameters that would have a constant architecture as the project included a demandable task.
The need for a different rectifier unit led to testing four other alternatives to ReLu. The one that was showed the best rates and consistency on the validation set loss was the Exponential Linear Unit (ELU) [14] as described by Clevert in his 2015 paper. The essential reason for choosing this activation function is the fact that both Leaky ReLus and Parameterised ReLus also include negative values and do not ensure effective reduction of image noise. The advantage of ELUs is the analysis of the percentage of a phenomenon in the input while the lack of that phenomenon is not mapped quantitevely. This also provides a generalisation method for the entire network, that in deemed necessary in the case of using synthetic data for training. The equation that the activation function uses is shown at (7).

$$f_{(x)} = \begin{cases} \alpha \times (e^x - 1) & for \quad x < 0 \\ x \quad for \quad x \geq 0 \end{cases}, \qquad f'_{(x)} = \begin{cases} 1 \quad for \quad x < 0 \\ f_{(x)} + \alpha \quad for \quad x \geq 0 \end{cases} \tag{7}$$
$$\text{where } \alpha = 0.001$$

As it is observed, by including a constant, ELU can saturate based on that value, when the inputs are negative. This also allows for the activation to adequately deal with the vanishing gradient problem. The main contrast from the other rectifiers is that ELUs takes negative values that push the gradient closer to zero. This also provides a clear distinction for the activation in terms of the instances in which it is active and inactive. ELUs can also limit the gap between the unit gradient and the normal gradient, improving the training time rate. This feature is also the key component of the batch normalization, that will be discussed in the following section. This clear saturation that is introduced with this function allows the rectifier to outperform the previously suggested methods in tests that were done in a variety of different task (such as Optical Character Recognition with the MNIST dataset, as well as the use of a 15-layer CNN on the ImageNet) [14]. For this dataset, ELUs are found to be singly slower than other activations, mainly due to the fact that the hyperbolic function that is been utlised for negative values, instead of the linear equation found in LReLu and PReLu. However, the influence of the activation function is considered to be minimal during training, especially when the number of iterations is extensive and the volume of data is substantial [40].

4.3 Performing normalisation at each training batch

A significant and considerable limitation of large-scaled neural networks is the fact that layer's input are modified during each training iteration, as the parameters of the previous layer that the current layer is connected to change. The outcome of this is slower rates during training and produces difficult to train models with the nonlinearities saturated. This problem is named as the internal covariance shift and perceived as the variation in the

activations of the model, in terms of their distribution, because of the continuous change of parameters during each training step.
A way of solving the internal covariance shift problem, as proposed by Ioffe and Szegedy [41], is the use of a normalisation technique for each layer's input. The essence of this method is that normalisation becomes an integral part of the entire model and thus performed as each batch of information is passed to the model, at every epoch of the training phase. This also implies faster training, since the stability of nonlinearly separable cases of inputs becomes stronger and the optimiser avoids remaining on the saturated condition. The gradient flow of the entire network can also be improved by this technique, since the dependence of gradients in parameters that are shifted at each epoch or to their initial values is significantly reduced. Additionally, the improvement minimises the risk of divergence for the deep network, without the compomisation on the learning rate of the system. This furthermore provides the means for a smoother Dropout [42] as randomising the outputs of one layer, that will be used on the next as inputs, becomes less important, due to the fact that normalisation is performed firstly. The initial technique used for Batch Normalisation [41] was the modifications of the inputs at each layer to achieve a constant distribution of features. This modification can be perceived as either creating a dependency to the activation values for the optimiser, or a direct change in the overall network structure [43][44]. It is salient to underline the importance of the network producing activations with designated distributions for parameter values used, as if the previously mentioned modifications (both forms of batch normalisation) intermixed with the optimisers, may require systematic updates on the normalisation values themselves, which drastically reduce the gradient decent impovement.
Because of the difficulty of the task, the normalisation function takes the form as found in (8) to deal with the instances in which some cases of non-differentiable type of inputs.

$$(x'^{(n)}) = \frac{x^{(n)} - E[x^{(n)}]}{\sqrt{Variance[x^{(n)}]}}, \quad where: \quad E[x^{(n)}] = \frac{1}{N} \times \sum_{i=1}^{N} x_i^{(n)} \tag{8}$$

As it is observed from the equation, the normalisation is equal the result produce by subtracting from the n-dimensional space layer input ($x = (x^{(1)}, x^{(2)}, ..., x^{(n)})$) the mean of all the values used in training with their respected dimensionality as well (n), devided by the square root of the unbiased variance estimate ($Variance[x^{(n)}] = \frac{m}{m-1}$), with a greater than one mini-batch size(m). This type also avoids the decorrelation of the image features through layers of the network. Normalising however, each input individually may case variations to what the network perceives. this can also gradually decrease the accuracy and thus caution in necessary when normalisation is performed. If the identity transform can be effectively represented by the transformation inserted in the network, the normalisation has been applied effectively. In a neural network that includes Batch Normalisation and is trained with it, each example in the training procedure is perceived concurrently with other examples of the batch. This allows the network to avoid deterministic values that are solely based on single instances for each example. For the requirements in the

current system the batch normalisation has been applied to every convolution layer, that its inputs is the product of a previous convolution layer (Layers 2-6) and additionally the outputs of the last convolution layer, before the data is flatten and the output of the hidden layer, as seen in Fig. 6. With this network structure that was proposed, the total number of parameters associated with normalisation, was pushed to 3840 from which half of them are trainable and the other half is not. As it could be seen, the Batch Normalisation method that was used shows many similarities to a proposed standardisation layer [45]. Although, Batch normalisation targets stable distributions during each pass of mini-batched through the network, while the normalisation layer is applied on-linearly to the output of the previous layer to achieve sparse activations.

4.4 Sample-based discretisation process

After the image features are extracted using convolution, they are to be used for the classification. Utilising the feature space in the entirety could be proven to be computationally challenging as if the dimensionality of the images through the network remain the same, the total number of parameters will increase sharply. Increasing the image complexity can also, show be a cause for the existence of overfitting. However, noting that convolution is used in the occasion that the classification task is performed in images, because that images are considered to be stationary sources of inputs, meaning that a informational region in one image, is likely to show significant information in another image.
Therefore, what is required is a method that can progressively reduce the overall complexity of inputs of network layers, by minimising the spatial size of the feature rich image. These layers are commonly referred to a *Pooling Layers* in which the input representations are down-sampled and become output of the layer. The process of pooling includes the use of aggregate statistics in specific parts of the image, that include distinct features. This normally is performed by using the mean or max value of an image to represent the feature with a decreased dimensionality. The pooling layer can be added to any part of the network as it acts independently on every depth slice of the image and resizes it appropriately. One of the most common pooling techniques is the utilisation of the maximum or mean value present in a region selected by the defined kernel. Thus the required variables that are to be defined when constructing a pooling layer are:

- The size of the filter to be used. Most filters in max pooling are either of size 2x2 or 3x3, as based on these values the kernel will traverse the entire image matrix respectively. Also taking to account the use of the mean or max value at each traverse, the system will compute and output the suitable value.
- The **stride** of the kernel, as it defines the step that is used while passing through the image vector. A larger stride will resolve in a smaller output, since less pixels will

overlap between kernel steps. For example a 2x2 filter with a stride of 2 will resolve in non-overlapping pixels in the final output down-scaled feature vector.

Conclusively, the pooling layer accepts an input of $W_1 \times H_1 \times D_1$ and the two hyperparameters for the filter size F_1 and the stride use S_1. They are used to produce the output feature vector of $W_2 \times H_2 \times D_2$ where $W_2 = \frac{(W_1 - F)}{S+1}$, $H_2 = \frac{(H_1 - F)}{S+1}$ while the depth remains the same since $D_1 = D_2$.
In this work between every two convolution layers a Max-pooling layer was added for decreasing the image dimensionality, as it has shown that it works better in practice than mean or average pooling. The kernel size was set to 2x2 with a stride of 2, for each pooling operation and therefore, the output will have half the width and half the height of the original image. In other words the first two convolutional layers will have 48x48 inputs, the next two will be 24 by 24 and the last two convolution layers will have inputs of 12 by 12, before the images are flatten to be passed to the network's hidden layer.

4.5 Regularisation of each layer

Because of the large number of parameters that the network architecture includes, as well as the complexity of the method. Overfitting is likely to occur. The final technique that is used to reinsure the avoidance of overfitting the training set is the use of Dropout [42]. As described in their paper by Srivastava and Hinton, Dropout is closely related to the natural theory of evolution and the reproduction system that exists in the nature. It does not only allow the most useful genes to be a part of a new generation in the population, but also reinsures the robustness of the genes if they are used coherently with another set of different genes. In neural networks, this translates to that every hidden unit is trained with a different set of neural networks through each training step, while still including the optimisation goal of achieving better accuracy rates and minimising the information loss though every epoch. Dropout is perceived as a stochastic regularisation metho. However, it can be managed in order to acquire deterministic regulisers. Dropout stochastically reduces the loss function by applying a noise distribution procedure.
Consider a neural network with H hidden layers, where we use h for an index $h \in \{1,2,\ldots,H\}$ and the two vectors for inputs and outputs are defined as $in^{(h)}$ and $out^{(h)}$. The weight of the layer is and the bias $W^{(l)}$ $b^{(l)}$ and the feed forward operation with Dropout is defined at (9) for every hidden unit i. Where $r^{(l)}$ is the vector of Bernoulli random variables, with probability of p being 1 and is used in conjunction with $in^{(h)}$ to create the thinned inputs by the Dropout function $\tilde{out}^{(h)}$.

$$\begin{gathered} in_i^{(h+1)} = W_i^{(h+1)} \times \tilde{out}^{(h)} + b_i^{(h+1)} \quad out_i^{(h+1)} = activation(in_i^{(h+1)}) \\ where, \quad \tilde{out}^{(h)} = r^{(h)} \times in^{(h)} \quad and \quad r^{(h)} \sim Bernoulli(p) \end{gathered} \tag{9}$$

Dropout can be utilised in neural networks with the use of gradient decent, similarly to the standard network with the exception that each batch of training examples is passed to a "thinned" version of the network, since some units were not used. Both forward and back propagation are done to the thinned version of the network. The gradients of each parameter are averaged over the course of the training steps, with parameters that do not contribute to the network (since they have been dropped) taking a gradient value of zero. In this network, Dropout was used four times, three of which were between the convolution layers in which the depth of the image is modified and before the images are flatten, while the last Dropout was before the output layer of the network. The Dropout at the convolution layers of the network is kept at 20% in order to minimise the probability of the network underlearning the data. The difference in the hidden layer and the Dropout that is applied to it's output is that 0.5 probability shows the maximum amount of regularisation [46], as it is the most important layer since it is responsible for determining the class of the image passed in the network. Also a 50% rate would yield an equal distribution for every network that is produced through each training step.

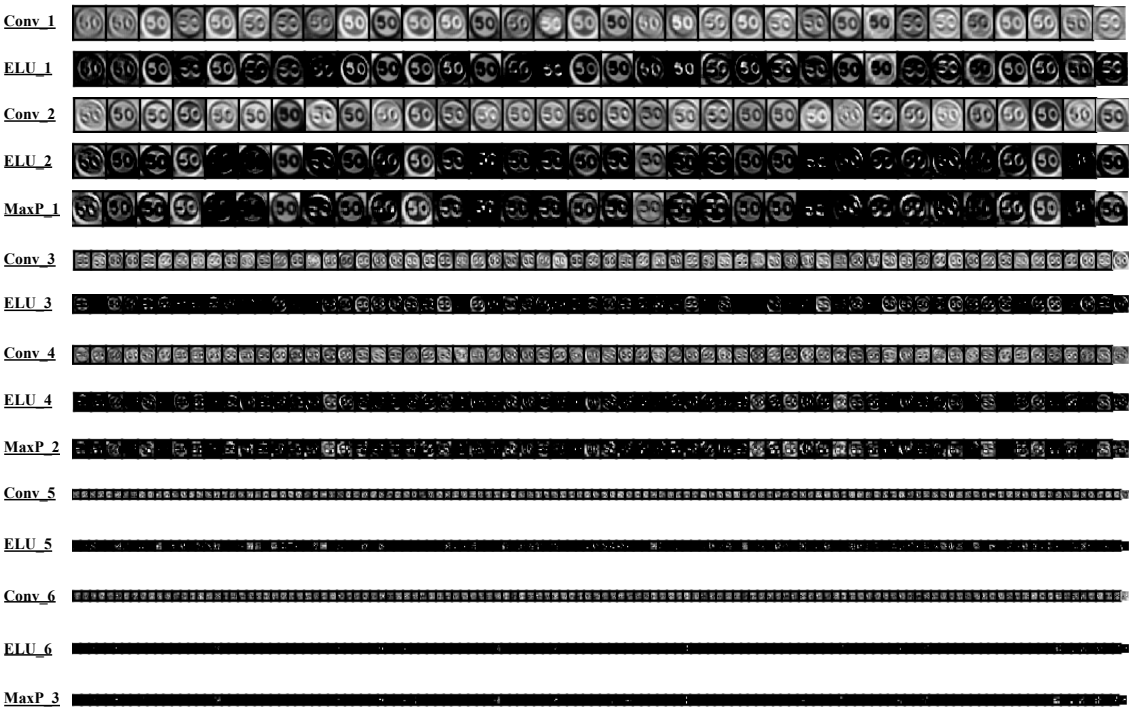

Figure 7: An example of what is perceived by the convolutional neural network at each layer. The image used is from the validation set with a Gaussian filter applied (4_42.jpg)

5 Experiments and Results

5.1 Background of experiments

The experiments performed with the use of generated data from training, were produced based on fifty different classes and applied twenty distinct Affine transformations. The final set was constructed from 1000 background images and four different brightness variations. The total images that the generator has created therefore is 2388397 with however, an uneven number of examples per class. The three signs with the most examples assigned to them were sign class 11 with 71037 examples, sign 22 with 67518 and sign 25 with 62717 examples. The main problem was the single greyscaled road sign (category 50), which in the generation process suffered significantly from the lack of usable images, as the space of variations was small compared to the other signs (since the other signs included three channels while this had only one). The number of examples that were created for class 50 was limited to 3414. For this reason, the experiments performed on samples from each of the categories that reinsure stratification of classes between the training and validation sets. The classifier was fed with 3400 images per class and the total number of training images reaching 170000. The validation was performed on 2500 real world images. Because of the magnitude of the SGTSD that was created, sampling based on human intuition was not possible to be done within the given timeframe, therefore the sample chosen for each sign is composed of randomly selected images. Although this does not guarantee that the limit in terms of accuracy rates is achieved, it can give a good approximation of the values that are expected to to be found, if the training was done an a specific sample that shows high quality examples, or the entire set of 2388397 images. Additional runs were performed with the use of different activation techniques and also with the use of smaller datasets. It is significant to mention that the training of the Convolutional Neural Network was performed on a standard 16GB CPU and not on a graphics card, thus the times that are shown on Table 2 are solely based on the capabilities of the current systems CPU, however these times would vary significantly if the system was run on a GPU.

Table 2: Tasks and their respected running times.

Task performed by the system	Time required in hours (aprox.)
SGTSD w/ 500000 images	6 hours
SGTSD - complete	**26 hours**
Normalisation of sets - SGTSD w/ 500000 images	1 hours
Normalisation of sets - complete SGTSD	**6 hours**
CNN run w/ 90000 images (50 epochs)	27 hours
CNN run w/ 170000 images (50 epochs)	50 hours
CNN run w/ 170000 images (100 epochs)	**103 hours**

Moreover, with the arrangement of resources and structure of the generator that was described, the number of all the possible images that were under consideration for their usability was 4000000. Nevertheless, because of the threshold values used to segment the information rich images, the number is down-scaled to 2388397. The initial images were created in a .PNG format to allow the use of four channel format for the images (to thus blend the foreground with the background in a new image). So the size that was required for their storage was close to 40 GB of storage space before they are downscaled to .JPEG format and the unusable signs are extracted and deleted, in order to the final set to be created. The sizes and values of each class are recorded on an automatically generated text file within the SGTSD. The file includes the number of examples per class, as well as the total number of useful images that the system has created and can be used for the training process. The "Read me" file that is also generated shows general specifications and information in regards to the process and the dataset. Tests only used the greyscaled versions of the images as it was suggested by LeCun [3] that the colour of the signs is not a decisive factor for the classification procedure, as greyscaled images showed competitive results compared to the coloured images. Furthermore, the greyscaled images are efficient in minimising the image noise that significantly varies between the training and validation data, therefore by eliminating a part of the increased dimensionality of the images, the synthetic examples provide better cases for training.
Since the images were greyscaled, the procedure for information extraction that was utilised was the histogram equation for each single-dimensional image that was fed to the network along side with the label of the sign. Nonetheless, if required by the human user, the system can be altered easily since the computation is directly performed on the image, thus an addition of three lines of code for defining each colour channel (considering that the colour model chosen has three values) will allow the use of an increased depth image.
The batch size that the network was trained with included 32 images. The creation of this mini-batch and its use for training is to limit the training time required by the network. This was based on the trade-off between the batch size and the number of iterations used to train the network. Nonetheless, more importantly, was because of the CPU limitations of the system and time that it could possibly require for each network epoch based on a larger batch size. An increase in the batch size may also show a decrease in the ability of the model to generalise (which is one of the most important aspects of this work, as the difference between real and synthetic sets are greater than in other similar systems) [47]. This is because of the tendency of large batches to become sharp minimizers when training. Thus, sharp minima consequently leads to poorer generalisation, when on the other hand, small batches converge to flat minimisers since there is an inherited noise in the gradient estimation process. This is also shown when taking the loss function, as large batches are heavily engaged in regions where a sharp minima exists, while on the contrary small batched are bound to small positive eigenvalues.

5.2 Classification results

The results, as presented, were obtained over the course of three weeks which included one day for the generator, one day for normalising the data and rest of the time for each of the models to train. The best accuracy rating that the classifier achieved was 91.84 %, while it remained constant during the later iterations of the training process close to 90% $\pm$ 1.5%. It was shown that by applying Batch Normalisation (BN), the model was able to generalise significantly better since the accuracy was boosted from 87.88%, that was outputted from a previous network version and with the use of ELU as a rectifier it reached 92.20%, with the same sample and number of epochs. Although initially as it was seen in Fig. 8 and Table 3, using LeakyReLu and PReLu, the accuracy rates were higher during a larger number of iterations than the ones achieved with ELU, the loss function remained very high for these two rectifiers (close to 1.5 while ELU showed a decrease with 1.0). Although the two activation functions were not efficiently tested on the new classifier, ELU shows great improvement when approximating the identity of an example in cases that are close to the origin because of its exponential nature, while both PReLu and LeakyReLu cannot. Although the difference in rectifiers show little difference in the runs performed, a more notable increase was shown with BN, as non-trainable parameters allowed the construction of a better model that can map the difference of real data as used for validation and synthetic for training.

Table 3: Accuracy rates and Kappa statistic based on activation and Batch Normalisation.

Classifier type	Accuracy rates (%)	Kappa statistic
CNN w/ LeakyReLU - 50 epochs	87.88%	0.8788
CNN w/ PReLu - 50 epochs	87.03%	0.8703
CNN w/ ELU - 50 epochs	87.88%	0.8788
CNN w/ ELU & BN - 50 epochs	**92.20%**	**0.9219**
CNN w/ ELU & BN - 100 epochs , new data	**91.84%**	**0.9183**

As shown by the quantitative results in Table 3, the model is capable of performing close to the lower and mid-nineties accuracy when the full system is used. Although the margin between the created system and others that a part of the current state-of-the art and utilise real data for training, is high, the outcomes are promising as it shows that with the correct sampling (difference between the two last runs), the margin can be minimised even more. In terms of the error cases as it can be seen by the truth table in Figure 9, the classes that display a lower true positive results by the model are the two classes (28 and 29) that are similar, as they both are warning signs associated with the narrowness of the road ahead ("narrow road on the right lane" and "narrow road on the left lane" respectively). This can be consider a rare case where the lack of high accuracy rates can be deemed less "harmful" for the overall performance of the classifier, since it shows that the model develops

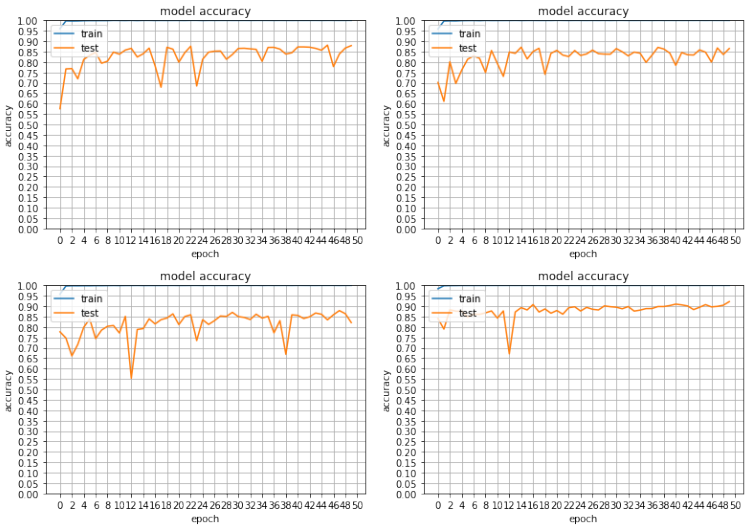

Figure 8: Accuracy rates with different activation functions and the improvement in results when introducing Batch Normalisation. Top right 2d vector shows the epochs of a model with LeakyReLu while the one on the left of it is trained with PReLu. The bottom two 2D vector use ELU as a rectifier with the exception that the bottom left also includes Batch Normalisation and shows a boots in rates as the model generalises more efficiently.

a deep understanding for each sign class and therefore, classes that display very limited variations are reasonable to be misclassified by the network. These cases can be difficult for even human users, as in the training data used, based on the fact that a Gaussian filter is used to eliminate the image noise, the differences between the two classes are almost insignificant for a human observer to understand. Another class that showed some lower than average recognition values was the loose chippings sign (class 14) which was misclassified 10% of the times as an intersection warning sign. Although, the similarities are less obvious for an observer, than in the previous case in which the error of the classifier could be categories as close to "human", both signs are in the same category as they are warning signs and it was calculated that the number of black pixels in both occasions are very similar (approximately the 7.7% of the sign for the intersection warning while 7.9% for the loose chipping sign). Additionally, the placement of the graphic on the inside area of the sign shows similarities in the sense that both allow some white margin from the overall red triangular border and they are also covering a larger than normal portion of the sign. In other words, because their orientation is not clearly defined, for example on the left, right, top or bottom of the sign's inside, they are closely linked, representation wise, by the convolutional neural network.
Another evaluation method that was used to understand the results of the classifier, with respect to the inherited difficulty of the task as well, was the kappa statistic that was cal-

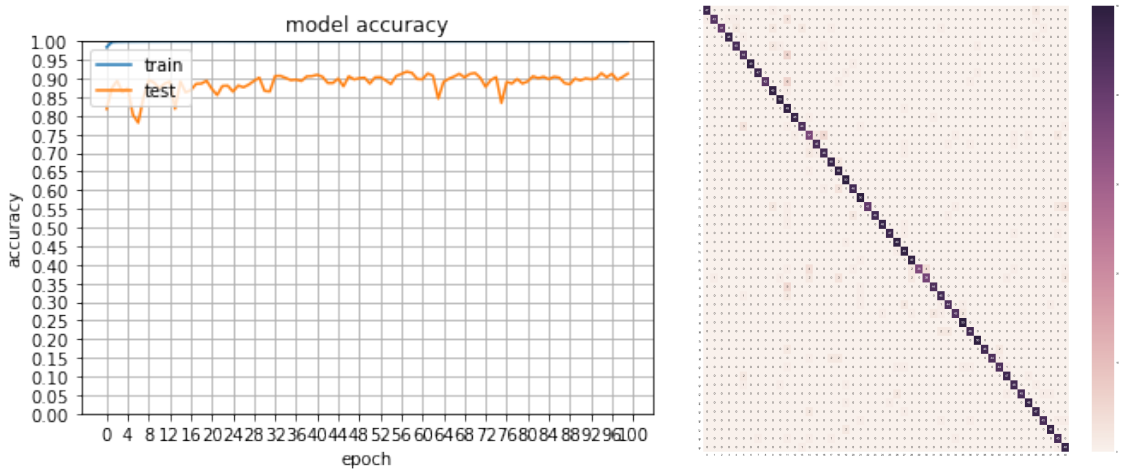

Figure 9: The accuracy rate achieved with a set of 170000 training examples (3400 per class), while the classifier was validated on a set of 2500 real world road sign examples.

culated for every model that was constructed. Cohen's kappa statistic is calculated based on the following type: $k = \frac{observed \quad accuracy - expected \quad accuracy}{1 - expected \quad accuracy}$. The expected accuracy of a randomly guessing classifier is significantly small (0.0239) and thus, the difficulty of the task and the number of classes that are used, make the creation of a valuable solution strenuous, for regular classifiers without advanced features. Additionally, based on the type and the expected accuracy calculated, as it is shown in Table 3, the kappa statistic of each model is very close to the observed accuracy of each individual system. Thus, the last two neural networks require only an improvement of approximately 7.2 – 6.8 % to be achieve a perfect predictor value. The utilisation of the kappa statistic coefficient was significant for this task since the overall difficulty of the task could be perceived with the computation of the expected accuracy. Also it enabled a demonstration of the validity of the assumptions that were made by the network, showing that they were not based on a random allocation of the classes, but rather creating connections because of an understanding of the representations presents in each image.

6 Summary, Conclusions and Future work

6.1 Summary and Conclusions about the project

Computer Vision has made large leaps over the last few years. One of the first objectives that have been intensively studied was the creation of efficient Advance Driving Assistance Systems (ADAS). Often cited papers that were published in the last four years have shown recognition rates that compete human accuracies and with the most recent ones achieving better than human recognition rates [27][6]. Putting into prospective these rates, only purely distorted images in which the signs may be disfigured and their shapes heavily altered, are shown to be the source of error rates as other that demonstrate a smaller degree of distortion are classified correctly by these systems.
However, this massive progress has risen new opportunities for the field of Autonomous Systems and in particular in the field of driverless systems as road sign recognition and classifications is an integrated part, when construction ADAS applications both in academia but also more intensively now in the industry. A common challenge that is faced by these projects is the fact that the road signs vary for country to country and thus, an adaptation of the dataset needs to be made. Alternatively, the entire classifier will need to be re-trained with the new set and re-evaluated. In this dissertation, we developed a model that is capable of not only efficiently classify the classes of real world traffic signs but to also create a generator function capable of creating the training set that is used (SGTSD). Our attempts were focused on the decrease of the need of a pre-defined set of training examples tailored to one countries road sign and the introduction of a set of template images, that can be user defined, to be used as the building block for generate synthetic data of variable size, which will then be available, to train the classifier as shown in Chapter 3. The methods to produce these results included not only the use of image blending, but also a simulation of the brightness of the background and an affine transformation function that would produce different poses for each traffic sign. It was argued that one of the drawbacks of this method was the fact that a portion, of the population of the generated data, was deemed uninformative and unsuitable as training examples. Therefore, a number of conditions were used based on threshold values that were found through testing, to eliminate this set of data and only include the images which are found to be good representations, in the final dataset. Thus, as discussed, the essential part of the generator process is the flexibility in the definition of the templates that can be used and also the number of different backgrounds that can be defined by the user in order to generate larger or consequently, smaller datasets and tailored to his/her needs.
For the classification process, the main idea was the use of Deep Learning and more noticeably Convolutional Neural Networks as the architecture of the classification model was analysed in Chapter 4. The network included six convolutional layers with the addition of one hidden layer that was used as the primary source for generating the probabilities

for each example to be a part of a class. In addition to these layers, a variety of advance activation functions were used in order to deal with the dying relu problem, with the one showing the best results from the tested rectifiers being the exponential linear unit (elu). Also for the network to normalise each layers input and thus deal with the problem cause by considering the variance in the layer input during each epoch, a batch normalisation technique was used. Finally, the classifier also decrease the dimensionality of the input through the passage of information in deeper convolution layers in the network with Max-Pooling and also introduced regularisation by randomly selecting the neurons that fire at each mini-batch pass (Dropout). The conjunction of these techniques and the current architecture of the network were able to create a stable system for the classification process. Before the analysis of the results, a background of the experiments was explained as there was a need of presenting the size of the dataset as well as the times required by every aspect of the system, in order to run the code at it's entirety. Moreover, the storing space requirements were also mentioned, as the initial dataset generator will require four channel images (therefore .PNG files), since a blending function will take place for both the background and foreground, to create the final images. The images that were passed to the classifier were greyscaled to allow the better generalisation of the system with a mini-batch of images used when the training process commences. The tests performed based on the different models that were used as well as the results that were created by each of the network's architecture and were explained in Chapter 5. Apart from the different outcomes based on the activation function that was selected at each instance, it was also viewed how Batch Normalisation at each convolution layer, allowed the model to generalise better and thus display higher scores. Additionally, independently from the structure differences, the overall difficulty of the task was discussed in terms of the number of classes that were selected for the project, as well as the examples that were included in each class. This was perceived through the expected accuracy rate which is the score based on a classifier randomly selecting the category for each example (with these data the rate was shown to be 0.0239, with a perfect classifier achieving 1.0). Based on Table 3, the variations between the accuracy and the kappa statistics were minimal, thus concluding that all the models were able to create substantial representations for each sign, even with the use of synthetic data as the training sample.

From a modelling prospective, the model falls in the wider category of Computer Vision with the generator utilising distinct image processing methods while the classifier offering a Deep Learning approach to the required task. All the function's parameters, in both of the programs, are based on tests that were performed and are tuned to produce adequate quality results with some margin for improvement.

Despite the rapid growth of the field of Computer Vision, some limitations still remain as to how data can be obtained and the effectiveness of creating new data from a very limited amount of resources. An illustration of this example can be found at Figure 10.

Figure 10: Examples of cropped training (top) and validation (bottom) data with optically and perceivably similar backgrounds. The amount of image distortion in real signs are significantly increased compared to the ones synthetically created by the generator.

The duplication of image noise is very difficult to be reproduced with the use of standard techniques as it is based on many factors and different between examples that may be viewed as similar by a human observer. Effective techniques that try to impersonate image noise walk towards what is perceived by us as imagination where an image that is already viewed becomes distorted with the introduction of noise visually dissimilar to the one that the original image was viewed with. Machine Learning has been thoroughly applied to this field and not only for Computer Vision applications, but also for Human-Computer interactions and data processing among others.

6.2 Future work and improvements

The core part of the desired was effectively achieved in this work as it was shown from the implementation techniques and the results provided that synthetic data could be utilised for traffic sign recognition. The rates showed great promise since a larger amount of variance between training and testing examples existed, as the two sets were obtained with different means. Additionally, the methodology that was followed for creating of the data was cheap in a sense that the main focus on how the different affine transformations that were processed can be merged with a chosen background in order for the final result to show the maximum possible degree of realism.
Nonetheless, because of the power requirements for running both the generator and the classifier, intensive testing could not be performed effectively in order to find the best accuracy that the current configuration can achieve. As it was explained, the results that were presented in Chapter 5 showed an approximation of the best rates that the system can produce. Therefore, if the work was to be extended what would be required would be a better machine with possibly a powerful GPU that would have a CUDA core and

therefore, allow the ConvNet to run on it instead of the standardised 16GB RAM that was used (it is suggested that the system should have as a minimum an NVIDIA GeForce GTX 760 Ti or any of the more advanced models that followed after). It is speculated that by running the classifier on a CUDA enabled GPU, the classification can be performed fifteen times faster than in a normal CPU (this may vary depending on the models) therefore, allowing more time towards testing the suitability of the network and fine-tuning parameters, as well as finding possible limitations of the current structure. Moreover, a larger memory space can also be used for the project as the model's accuracy is strongly correlated with the number of examples that was given during training. An even larger number of examples can show further improvement in the recognition (although after a certain limit of examples the addition of further data do not yield significant enhancements in the accuracy). Nonetheless, more training examples may also allow the CNN to better generalise and it is the most effective method in avoiding overfiting, as the data for the training samples become larger and better represents the entire population of possible instances.

Apart form the use of more advance systems for the testing of the generator and the classifier, as well as the creation of a larger dataset, additional and further implementation methods may also be necessary to achieve better accuracy rates while minimising the loss function. These methods would be based on supplementary techniques that can increase the realism of the synthetic images without requiring additional results to run. Although, the complexity of the generator model would increase and for their utilisation further hardware requirements will needed, the results that are possible to be produced, can significantly decrease the synthetic noise that the training images show. Further implementations are mostly targeted towards performing texture synthesis to better represent the materials of the traffic signs. The main problem that this approach would face if the fact that road signs are considered a "glossy" material as the incoming light of a given direction (might be from the sun or street lighting) is not reflected equally at the entirety of the material and not in one direction as well. This also means that the directions to which the light is reflected to may change when the optical perspective of the observer is changed. However, not in a major way. But the sign will have a reflective nature in the case where the light is from a car's headlights, as they are designed to be specular in that specific light hue. Moreover, in this instance, considering the different illumination possibilities that the images may have, an additional difficulty arises for the task. For example, considering an image that is evenly illuminated (such from the light of a car hitting the sign directly), it will show a more "natural state" of the sign. Yet, in a different lighting environment, for example if the sun hits the sign at a certain angle, the conditions of the colours of the sign a well as the illumination will be significantly different. Therefore, a deep understanding of the sign material needs to be established by the system in order to be able to compute the different outcomes that can be expected for applying light of different sources and angles at the sign. A building block for the task may be the work

of Schied [48] which deals with noisy inputs and produces luminance variance estimates based on temporal accumulation. Although the work deals mostly with image denoising, it is believed that by using vague representations of the final samples to be used, utilitarian images can be produced. Furthermore, another work that has shown to be useful for synthetic image generation would the one of Aittala and Aila [49] with their system that creates an estimate of the material based on the flash of the input when rendered from different angles.
The last methodology that may be followed, as it was the alternative from the sampling procedure chosen, was the use of an Autoencoder that would be trained on synthetic data that are shown to be very good approximations from the dataset created. This would allow for creating a set that is closer to real data. However, for this process, a human expert would be required as it would be necessary to separate the images that are shown to be more informative examples and more complete representations of real data. Avoiding this would require further research in order to find a similarity method for images between the training and validation sets and eliminate the need of user involvement.

References

[1] M.-Y. Fu and Y.-S. Huang, “A survey of traffic sign recognition,” 2010 International Conference on Wavelet Analysis and Pattern Recognition, 2010.

[2] J. Miura, T. Kanda, and Y. Shirai, ”An active vision system for on-line traffic sign recognition,” IEICE TRANSACTIONS on Information and Systems vol. 85, no. 11, pp. 1784-1792, 2002.

[3] P. Sermanet and Y. Lecun, “Traffic sign recognition with multi-scale Convolutional Networks,” The 2011 International Joint Conference on Neural Networks, 2011.

[4] S. Salti, A. Petrelli, F. Tombari, N. Fioraio, and L. D. Stefano, “Traffic sign detection via interest region extraction,” Pattern Recognition, vol. 48, no. 4, pp. 1039–1049, 2015.

[5] Y.-Y. Nguwi and A. Z. Kouzani, “Detection and classification of road signs in natural environments,” Neural Computing and Applications, vol. 17, no. 3, pp. 265–289, 2007.

[6] J. Jin, K. Fu and C. Zhang, ”Traffic sign recognition with hinge loss trained convolutional neural networks,” IEEE Transactions on Intelligent Transportation Systems, vol. 15 issue 5, pp.1991-2000, 2014.

[7] F. Zaklouta, B. Stanciulescu, and O. Hamdoun, “Traffic sign classification using K-d trees and Random Forests,” The 2011 International Joint Conference on Neural Networks, 2011.

[8] J. Stallkamp, M. Schlipsing, J. Salmen, and C. Igel, “The German Traffic Sign Recognition Benchmark: A multi-class classification competition,” The 2011 International Joint Conference on Neural Networks, 2011.

[9] S. M. Bascón, J. A. Rodríguez, S. L. Arroyo, A. F. Caballero, and F. López-Ferreras, “An optimization on pictogram identification for the road-sign recognition task using SVMs,” Computer Vision and Image Understanding, vol. 114, no. 3, pp. 373–383, 2010.

[10] F. Larsson and M. Felsberg, “Using Fourier Descriptors and Spatial Models for Traffic Sign Recognition,” Image Analysis Lecture Notes in Computer Science, pp. 238–249, 2011.

[11] R. Timofte, K. Zimmermann, and L. V. Gool, “Multi-view traffic sign detection, recognition, and 3D localisation,” 2009 Workshop on Applications of Computer Vision (WACV), 2009.

[12] B. Xu, N. Wang, T. Chen and M. Li, ”Empirical evaluation of rectified activations in convolutional network,” 2015.

[13] K. He, X. Zhang, S. Ren, and J. Sun, “Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification,” 2015 IEEE International Conference on Computer Vision (ICCV), 2015.

[14] D. A. Clevert, T. Unterthiner, and S. Hochreiter, ”Fast and accurate deep network learning by exponential linear units (elus),” 2015.

[15] G. Klambauer, T. Unterthiner, A. Mayr and S. Hochreiter, ”Self-Normalizing Neural Networks,” 2017

[16] H. Akatsuka and S. Imai, “Road Signposts Recognition System,” SAE Technical Paper Series, Jan. 1987.

[17] M. D. S. Blancard, “Road Sign Recognition: A Study of Vision-based Decision Making for Road Environment Recognition,” Springer Series in Perception Engineering Vision-based Vehicle Guidance, pp. 162–172, 1992.

[18] F. Zaklouta and B. Stanciulescu, “Real-time traffic sign recognition in three stages,” Robotics and Autonomous Systems, vol. 62, no. 1, pp. 16–24, 2014.

[19] C. Yao, F. Wu, H.-J. Chen, X.-L. Hao, and Y. Shen, “Traffic sign recognition using HOG-SVM and grid search,” 2014 12th International Conference on Signal Processing (ICSP), 2014.

[20] E. Alpaydin, Introduction to machine learning. Cambridge: MIT Press, ch. 13, sec. 13.1-13.6, pp. 305–317, 2010.

[21] G. C. Cawley and N. L. C. Talbot, ”Preventing over-fitting in model selection via Bayesian regularisation of the hyper-parameters,” Journal of Machine Learning Research, vol. 8, pages 841-861, 2007.

[22] D. Cireşan, U. Meier, J. Masci, and J. Schmidhuber, “Multi-column deep neural network for traffic sign classification,” Neural Networks, vol. 32, pp. 333–338, 2012.

[23] Y. LeCun, L. Bottou, Y. Bengio and P. Haffner, ”Gradient-based learning applied to document recognition,” Proceedings of the IEEE, vol. 86, issue 11, pp.2278-2324, 1998.

[24] D. Scherer, A. Müller, and S. Behnke, “Evaluation of Pooling Operations in Convolutional Architectures for Object Recognition,” Artificial Neural Networks – ICANN 2010 Lecture Notes in Computer Science, pp. 92–101, 2010.

[25] N. Ueda, “Optimal linear combination of neural networks for improving classification performance,” IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 22, no. 2, pp. 207–215, 2000.

[26] U. Meier, D. C. Ciresan, L. M. Gambardella, and J. Schmidhuber, “Better Digit Recognition with a Committee of Simple Neural Nets,” 2011 International Conference on Document Analysis and Recognition, 2011.

[27] M. Haloi, ”Traffic sign classification using deep inception based convolutional networks,” 2016 IEEE Intelligent Vehicles Symposium, 2016.

[28] C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke and A. Rabinovich, ”Going Deeper with Convolutions,” 2015 Computer Vision and Pattern Recognition (CVPR), 2015.

[29] “Traffic-sign images,” Traffic-sign images - YOUGOV.UK. [Online]. Available: https://www.gov.uk/guidance/traffic-sign-images.

[30] Know your traffic signs, London: H.M.S.O., 1975.

[31] M. Berger, Geometry I, Berlin: Springer, 1987.

[32] A. Jain, Fundamentals of Digital Image Processing, Prentice-Hall, 1986.

[33] T.G. Connolly and W. Sluckin, Measures of Variability, In An Introduction to Statistics for the Social Sciences, Palgrave Macmillan, UK, pp. 41-57, 1971.

[34] “HSP Color Model - Alternative to HSV (HSB) and HSL,” HSP Color Model - Alternative to HSV (HSB) and HSL. [Online]. Available: http://alienryderflex.com/hsp.html.

[35] A. Buades, B. Coll, and J.-M. Morel, ”Non-Local Means Denoising,” Image Processing On Line, 2011.

[36] Y.L Liu, J. Wang, X. Chen, Y.W. Guo, and Q.S. Peng, ”A robust and fast non-local means algorithm for image denoising,” Journal of Computer Science and Technology, vol. 23, no. 2, pp.270-279, 2008.

[37] A. Krizhevsky, I. Sutskever and G.E. Hinton, ”Imagenet classification with deep convolutional neural networks,” In Advances in neural information processing systems, pp. 1097-1105, 2012.

[38] M.D. Zeiler and R. Fergus, ”Visualizing and understanding convolutional networks,” In European conference on computer vision, pp. 818-833, 2014.

[39] Y. Bengio, Learning deep architectures for AI. Boston: Now, 2009.

[40] Y. Jia, Learning semantic image representations at a large scale, University of California, Berkeley, 2014.

[41] S. Ioffe and C. Szegedy, ”Batch normalization: Accelerating deep network training by reducing internal covariate shift,” In International Conference on Machine Learning, pp. 448-456, 2015.

[42] N. Srivastava, G.E. Hinton, A. Krizhevsky, I. Sutskever and R. Salakhutdinov, ”Dropout: a simple way to prevent neural networks from overfitting,” Journal of machine learning research, vol. 15, no. 1, pp.1929-1958, 2014.

[43] D. Povey, X. Zhang and S. Khudanpur, ”Parallel training of deep neural networks with natural gradient and parameter averaging”, Computing Research Repository (CoRR), 2014.

[44] S. Wiesler, A. Richard, R. Schluter and H. Ney, ”Mean-normalized stochastic gradient for large-scale deep learning,” In Acoustics, Speech and Signal Processing (ICASSP), IEEE International Conference (pp. 180-184), 2014.

[45] Ç Gülçehre, and Y. Bengio, ”Knowledge matters: Importance of prior information for optimization,” Journal of Machine Learning Research, vol. 17, no. 8, pp.1-32, 2016.

[46] P. Baldi and P.J. Sadowski, ”Understanding dropout,” In Advances in Neural Information Processing Systems, pp. 2814-2822, 2013.

[47] N.S. Keskar, D. Mudigere, J. Nocedal, M. Smelyanskiy and P.T.P. Tang, ”On large-batch training for deep learning: Generalization gap and sharp minima,”5th International Conference on Learning Representations (ICLR 2017), 2016.

[48] C. Schied, A. Kaplanyan, C. Wyman, A. Patney, C.R.A. Chaitanya, J. Burgess, S. Liu, C. Dachsbacher, A. Lefohn and M. Salvi, ”Spatiotemporal variance-guided filtering: real-time reconstruction for path-traced global illumination,” In Proceedings of High Performance Graphics (p. 2), ACM, 2017.

[49] M. Aittala, T. Aila and J. Lehtinen, ”Reflectance modeling by neural texture synthesis,” ACM Transactions on Graphics (TOG), vol. 35, no. 4, p. 65, 2016.